CS771 : Introduction to Machine Learning

Minor Assignment-1

Aarsh Jain
230015
aarshjain23@iitk.ac.in

Aayush Kumar
230027
aayushk23@iitk.ac.in

Bikramjeet Singh
230298
bsingh23@iitk.ac.in

Hitarth Kirankumar Makawana
230479
hitarthkm23@iitk.ac.in

Kulshreshth Chikara
230586
kulshresh23@iitk.ac.in

Ronav Puri
230815
ronavgp23@iitk.ac.in

This report contains solutions to Minor Assignment-1 for the course CS771 2025-26 Semester-I for the group TheSinisterSix. It addresses the corrections made to the provided implementation of the SDCM algorithm for solving C-SVM. Following the error correction, an analysis of the influence of various hyperparameters was performed.

1 Theoretical Background

We can solve the Support Vector Machine (SVM) problem (without bias) either by solving the primal version

$$\underset{\mathbf{w}\in\mathbb{R}^d}{\operatorname{argmin}} \left\{ \tfrac{1}{2}\|\mathbf{w}\|_2^2 + C \cdot \sum_{i=1}^{n} \left[1 - y^i \cdot \mathbf{w}^\top \mathbf{x}^i\right]_+ \right\}, \tag{1}$$

where $[z]_+ = \max\{0, z\}$ denotes the hinge loss.

Alternatively, we may solve the corresponding dual formulation

$$\underset{\boldsymbol{\alpha}\in\mathbb{R}^n}{\operatorname{argmax}} \left\{ \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y^i y^j \langle \mathbf{x}^i, \mathbf{x}^j \rangle \right\} \quad \text{s.t.} \quad \alpha_i \in [0, C]. \tag{2}$$

To solve either of these formulations, one may employ optimization methods such as gradient descent, coordinate descent, or stochastic methods. In this analysis, we are concerned with solving the dual version through Stochastic Dual Coordinate Maximization (SDCM).

Concentrating only on the terms that involve a single coordinate α_i in the original equation (2), we obtain

$$\underset{\alpha_i\in\mathbb{R}}{\operatorname{argmax}} \left\{ \alpha_i - \tfrac{1}{2}\alpha_i^2 \|\mathbf{x}^i\|_2^2 - \alpha_i y^i \cdot \sum_{j\neq i} \alpha_j y^j \langle \mathbf{x}^i, \mathbf{x}^j \rangle \right\} \quad \text{s.t. } \alpha_i \in [0, C]. \tag{3}$$

Renaming $x = \alpha_i$, $q = \|\mathbf{x}^i\|_2^2$, and $p = y^i \cdot \sum_{j\neq i} \alpha_j y^j \langle \mathbf{x}^i, \mathbf{x}^j \rangle$, this reduces to

$$\underset{x}{\operatorname{argmin}} \quad \tfrac{1}{2} q x^2 - x(1-p) \quad \text{s.t. } x \in [0, C]. \tag{4}$$

The solution is obtained by first finding the unrestricted minimizer

$$\tilde{x} = \frac{1-p}{q}. \tag{5}$$

If $\tilde{x} \in [0, C]$, the solution is $\tilde{x}$; if $\tilde{x} < 0$, the solution is 0; otherwise the solution is C.

2 Solutions

2.1 Coding Errors in Cell 5

In this part, we have listed all the errors present in the cell 5 of the implementation along with their justifications.

- **Error 1: Incorrect Projection of α_i:**

  The projection of α_i in the original code is incorrect:

  ```
  if newAlphai < C:
      newAlphai = C
  if newAlphai < 0:
      newAlphai = 0
  ```

  This is because any value of `newAlphai` < C is forced to C, but as per equation (3), $\alpha_i \in [0, C]$. So the corrected version should instead project values below 0 to 0 and values above C to C else no changes:

  ```
  if newAlphai < 0:
      newAlphai = 0
  elif newAlphai > C:
      newAlphai = C
  ```

  This ensures `newAlphai` $\in [0, C]$ as required by the dual constraints.

- **Error 2: Incorrect bookkeeping update for w_{SDCM} and b_{SDCM}:**

  We maintain the parameters from the dual as

  $$w_{\text{SDCM}} = \sum_{i=1}^{n} \alpha_i y^i x^i \qquad b_{\text{SDCM}} = \sum_{i=1}^{n} \alpha_i y^i$$

  where the bias is treated as the *hidden last coordinate* of the weight vector having one extra dimension (the corresponding feature for the hidden bias in X is 1). When only α_i changes from α_i^{old} to α_i^{new}, the correct $O(d)$ bookkeeping is

  $$\Delta \;=\; (\alpha_i^{\text{new}} - \alpha_i^{\text{old}})\, y^i \qquad w_{\text{SDCM}} \leftarrow w_{\text{SDCM}} + \Delta\, x^i \qquad b_{\text{SDCM}} \leftarrow b_{\text{SDCM}} + \Delta$$

  In the original code, instead of removing the old value and adding the new one it goes on to remove both of them:

  ```
  w_SDCM = w_SDCM - (newAlphai + alpha[i]) * y[i] * x
  b_SDCM = b_SDCM - (newAlphai + alpha[i]) * y[i]
  ```

  This leads to wrong bookkeeping of the concerned variables resulting in drastic miscalculations. Instead the corrected code should look like:

  ```
  w_SDCM = w_SDCM + (newAlphai - alpha[i]) * y[i] * x
  b_SDCM = b_SDCM + (newAlphai - alpha[i]) * y[i]
  ```

  This uses the difference $(\alpha_i^{\text{new}} - \alpha_i^{\text{old}})$ to first remove the old contribution and then add the new one, consistent with $w_{\text{SDCM}} = \sum_i \alpha_i y^i x^i$ and $b_{\text{SDCM}} = \sum_i \alpha_i y^i$ (the latter corresponding to the bias with feature value 1).

- **Error 3: Missing function definitions for $getCyclicCoord$ and $getRandCoord$:**
  (Not an error contributing to the inaccuracy of the code)

  The driver function `doSDCM` takes in an argument `getCoordFunc` that references a function `coordinateGenerator(mode, d)` that selects the coordinate for SDCM and returns it alongwith some state for future calculation. Here, mode refers to the types of coordinate selection namely `cyclic, randperm` and `random`.

  The modes `cyclic` and `random` were being referenced in the code but the helper functions weren't defined. This leads to a runtime `NameError` as soon as either mode is selected.

Added definitions (to resolve the missing functions):

```
def getCyclicCoord( state ):
  idx = state[0]
  d = state[1]
  if idx >= d - 1 or idx < 0:
    idx = 0
  else:
    idx += 1
  state = (idx, d)
  curr = idx
  return (curr, state)

def getRandCoord( state ):
  idx = np.random.randint( 0, state )
  state = state
  curr = idx
  return (curr, state)
```

- **Error 4: Off-by-one error in coordinate generators (initial state handling):**
  (Contributes to incorrect iteration order but does not cause a signifcant accuracy change)

  In the helper functions `getCyclicCoord` and `getRandpermCoord`, the initial state was being set to `(0, d)` or `(0, perm)`. With this initialization, the very first element (`0` in cyclic mode, or the first element of the random permutation in randperm mode) was skipped because the update rule increments the index before returning it.

  Fix: Initialize the index with `-1` instead of `0`. This ensures that the first call correctly resets the index to 0 (or the first element of the permutation) and subsequent iterations proceed without skipping any element.

```
# Cyclic generator
elif mode == "cyclic":
    return (getCyclicCoord, (-1, d))

# Randperm generator
elif mode == "randperm":
    return (getRandpermCoord, (-1, np.random.permutation(d)))
```

2.2 Test Accuracy and Primal–Dual Objective Curves

With the corrected implementation of SDCM and default hyperparameters, the results are:

- **Final test accuracy: 95.90**%
- **Observation:** Accuracy is consistent with expectations after fixing the update logic.

Primal and dual objective curves. Figure 1 shows the primal and dual objective values plotted against cumulative time. The dual objective steadily increases while the primal decreases, and the gap between them narrows, indicating convergence.

This also validates the *weak-duality* theorem that the the dual function is always lower than that of the dual function and they both meet in the special case of strong duality (which is the case here since our function is *"convex and nice").*

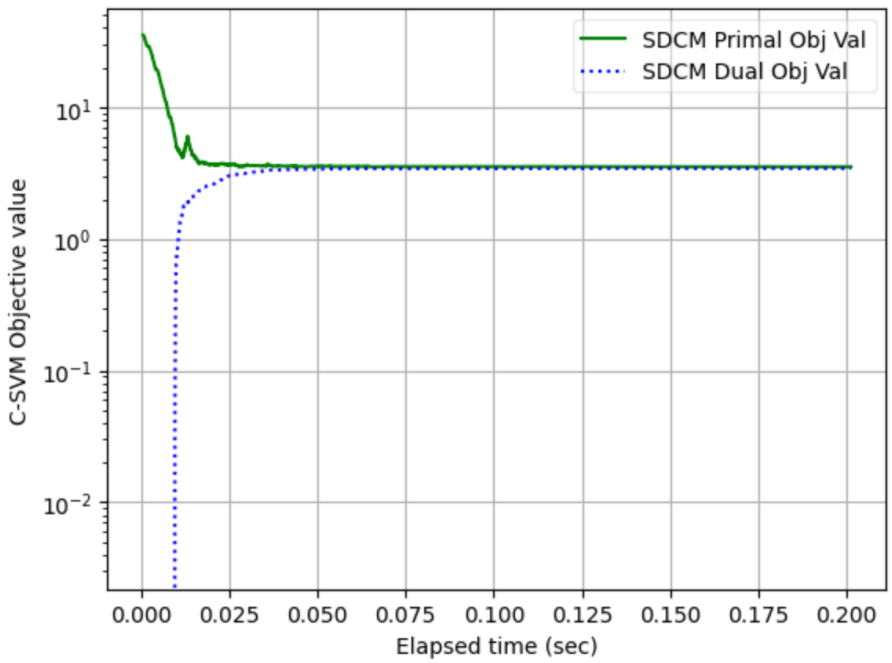


Figure 1: Primal and dual objective values vs. cumulative time (corrected SDCM, default hyperparameters).

2.3 Effect of the Penalty Parameter C

From the table and the plots given in Figure 2, we discuss how C influences both accuracy and primal–dual convergence.

Table 1: Effect of varying C on test accuracy.

C	Test Accuracy (%)
10^{-5}	91.96
10^{-4}	91.96
10^{-3}	92.59
1	96.46
10^{3}	92.81
10^{4}	78.47

Observations From the plots, it is observable that very small values of C lead to flattening of the primal–dual objectives and there is drop in the accuracy as compared to the initial C value (e.g. 0.01), while very large values of C cause unstable and slow convergence of the objectives. In contrast, moderate values of C achieve a balance between margin maximization and error penalization, yielding smooth primal–dual convergence and the best generalization accuracy.

2.4 Hyperparameter Studies

2.4.1 (a) Coordinate generator: `randperm` vs `cyclic` vs `random`

Setup. To answer this part of the question, I focused on the effect of changing the coordinate generator inside the corrected `doSDCM` implementation. The generator decides which α_i gets updated at each iteration. By default, the implementation uses `randperm`, which shuffles all coordinates and visits them once per pass.

To compare, we also tested two other strategies:

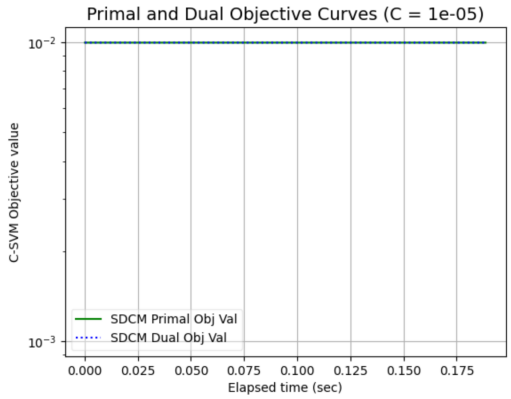
- `cyclic`: updates the coordinates in a fixed round-robin order $1 \to n \to 1$.
- `random`: chooses a coordinate completely at random at each iteration (some may get repeated, some may be skipped for a while).

All three methods (`randperm`, `cyclic`, and `random`) were run under the same setup so that the results are directly comparable. I fixed $C = 0.01$, set the horizon to $T = 20000$ iterations, initialized all parameters to zero, and used the same train/test split. For each run, I recorded both the final test accuracy and the convergence behavior of the primal and dual objectives over time.

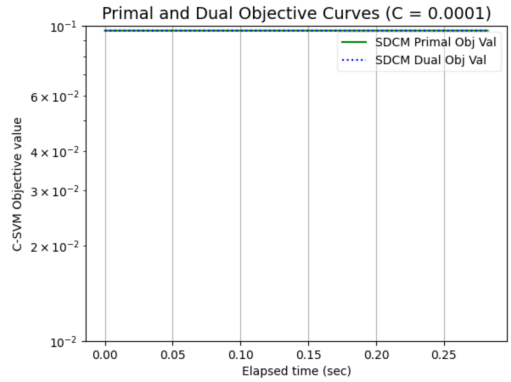
Results. Table 2 shows the final test accuracies and a qualitative summary of convergence behavior. Figure 3 plots the dual and primal objectives over time for all three generators.

Table 2: Effect of coordinate generator on test accuracy and convergence.

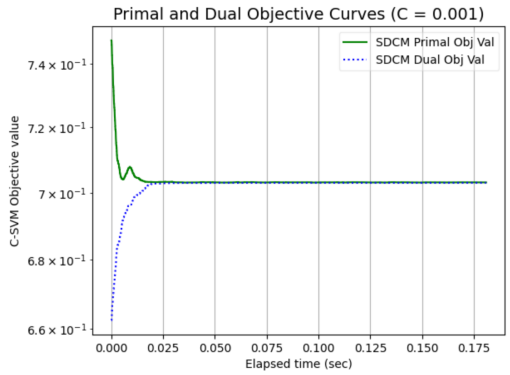
Generator	Test Accuracy (%)	Dual Convergence	Primal Convergence
`randperm`	95.46	smooth, steady	stable
`cyclic`	95.26	very smooth	stable
`random`	95.36	noisy, fluctuating	slightly unstable



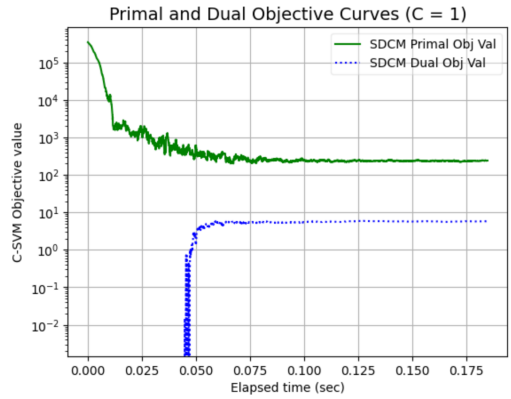
(a) $C = 10^{-5}$



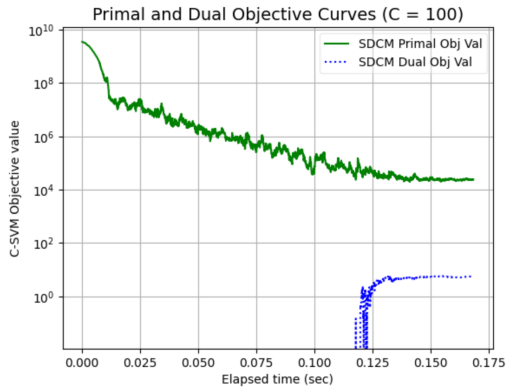
(b) $C = 10^{-4}$



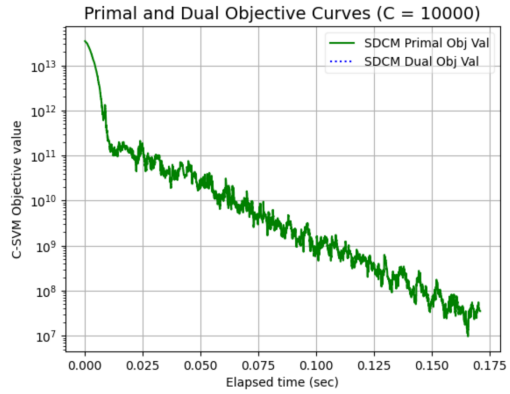
(c) $C = 10^{-3}$



(d) $C = 1$

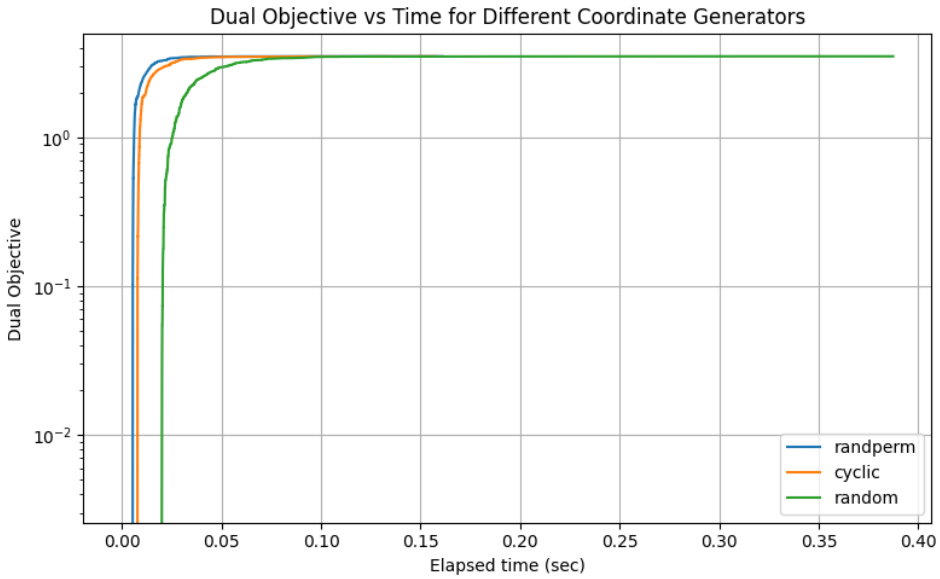


(e) $C = 10^{2}$

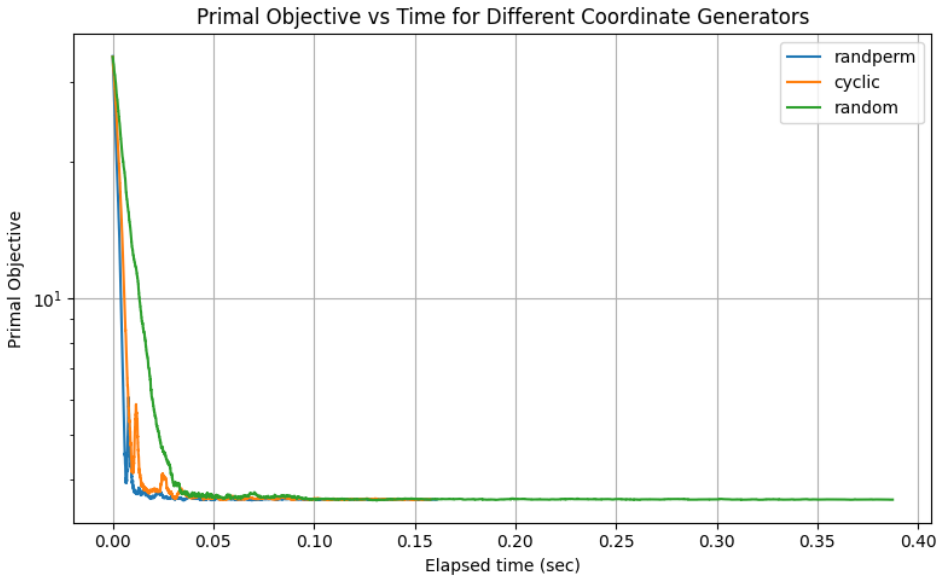


(f) $C = 10^{4}$

Figure 2: Primal and dual objective values vs. time for different values of C.



(a) Dual objective vs. time for different coordinate generators.



(b) Primal objective vs. time for different coordinate generators.

Figure 3: Comparison of primal and dual objective values over time for different coordinate generators (`randperm`, `cyclic`, `random`).

Observations.

- `randperm` gave fast and reliable convergence. Its stochastic reshuffling avoids bias while still ensuring each coordinate is updated exactly once per pass. Accuracy was highest at 95.46%.
- `cyclic` produced the smoothest convergence curves, since coordinates are updated in a strict order. Accuracy (95.26%) was very close, though early convergence can be slower if "hard" coordinates are updated late in the cycle.
- `random` was the noisiest: some coordinates were updated repeatedly while others were skipped, making the curves slightly unstable. Its accuracy was 95.36%, competitive but not as strong as `randperm`.
- Overall, the coordinate generator mainly affects the *stability and smoothness* of primal–dual convergence. The final test accuracy remains high across all three modes (95.2%–95.5%).

2.4.2 (b) Initialisation: all-C vs all-0 vs random

Setup. To study the effect of initialisation, I compared three different schemes for the dual variables α in the corrected SDCM implementation:

- **all-C:** initialise every α_i at the upper bound C.
- **zeros:** initialise every $\alpha_i = 0$.
- **random:** initialise each α_i independently from a uniform distribution on $[0, C]$.

All three schemes were tested under the same setup to ensure direct comparability. I fixed $C = 0.01$, set the horizon to $T = 20000$ iterations, used the `randperm` coordinate generator, and the same train/test split. For each run, I recorded the primal and dual objective values, the primal–dual gap, and the final test accuracy.

Results. Table 3 shows the final test accuracies for the three initialisation schemes. Figure 4 plots the primal–dual gap convergence over time.

Table 3: Effect of initialisation on test accuracy and convergence.

Initialization	Test Accuracy (%)	Convergence Behavior
all-C	95.00	larger initial gap, converges steadily
zeros	95.02	smallest initial gap, smooth convergence
random	95.11	slightly noisy early phase, then stable

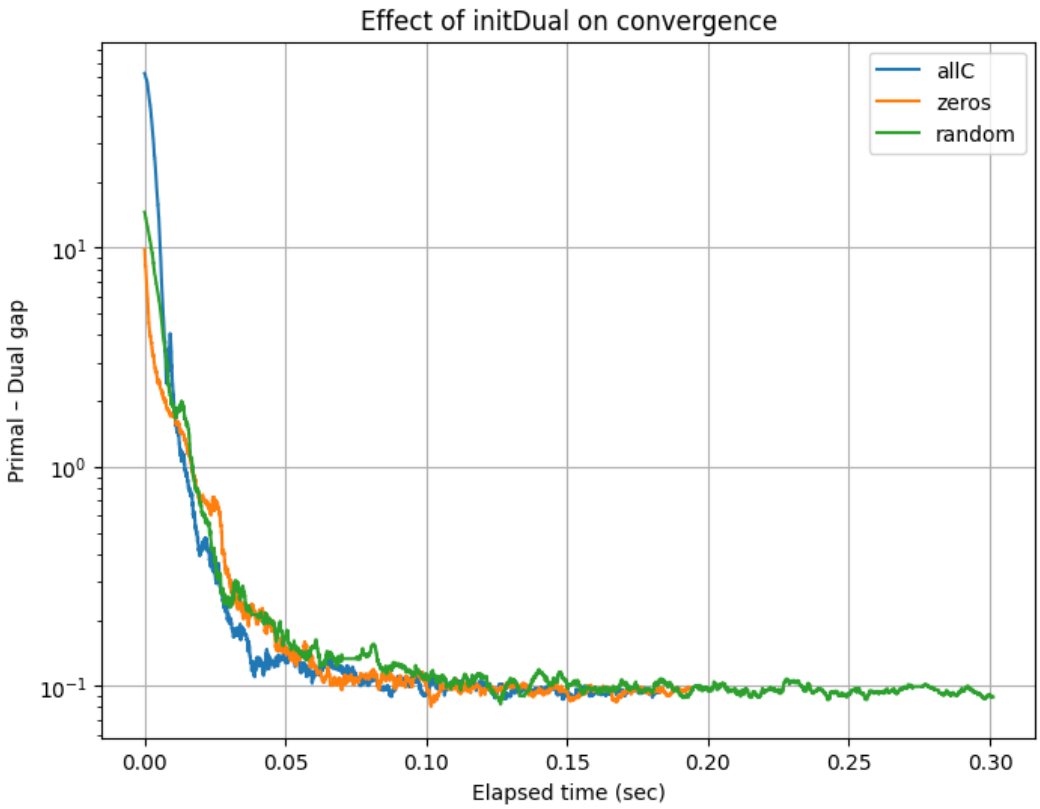


Figure 4: Convergence of primal–dual gap under different initialisations (all-C, zeros, random)

Observations.

- **All-C** starts furthest from the optimum, with a large primal–dual gap in the early iterations, but eventually converges to the same level as the other inits.
- **Zeros** gives the cleanest and fastest convergence; the primal–dual gap decreases smoothly from the beginning.
- **Random** behaves in between the two extremes, with some run-to-run variability and noisier curves early on, but it stabilizes and achieves slightly higher test accuracy in this run.

Overall, the choice of initialization mainly affects the *early convergence speed and stability* of the primal–dual gap, while the final test accuracy remains consistently high ($\approx 95\%$) across all schemes.

2.4.3 (c) Horizon (number of iterations)

Setup. The baseline solver `doSDCM` is run for 20000 iterations. To study the effect of the horizon, we varied the maximum number of iterations while keeping all other hyperparameters fixed. Specifically, we experimented with horizons of $\{1000,\ 5000,\ 10000,\ 20000,\ 50000,\ 100000\}$.
For each horizon, we trained the model and compared:

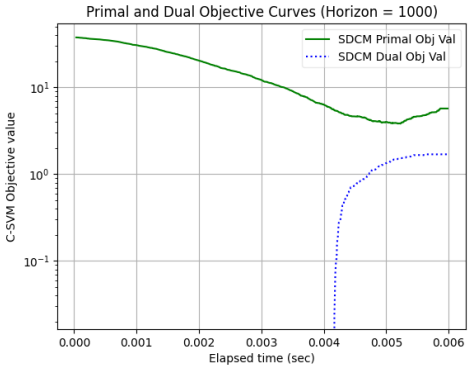
- the final test accuracy,
- the primal/dual objective curves across iterations.

Results. Table 4 reports the test accuracy for different horizons (number of iterations). We observe that increasing the horizon leads to higher accuracy. However, the rate of improvement slows down as the horizon becomes large, and the accuracy eventually saturates at a near-constant value. Conversely, when the horizon is reduced significantly, the algorithm terminates too early, and the solution gets significantly worse.

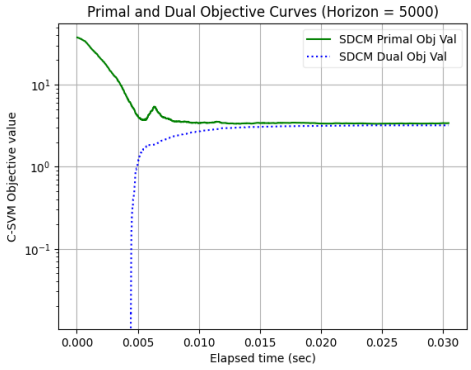
Table 4: Effect of varying the horizon on test accuracy.

Horizon	Test Accuracy (%)
1000	84.84
5000	94.56
10,000	95.15
20,000	95.41
40,000	95.40
80,000	95.42

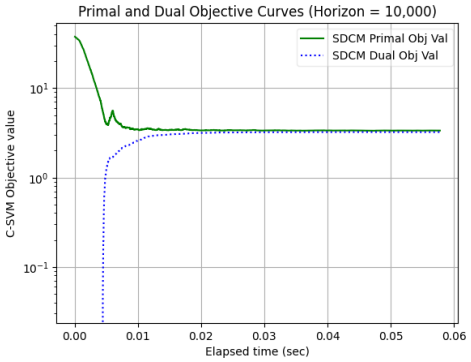
As shown in Figure 5, the convergence behavior improves with a higher horizon. Fewer iterations lead to noisier and slower convergence, whereas more iterations allow the algorithm to stabilize more quickly.



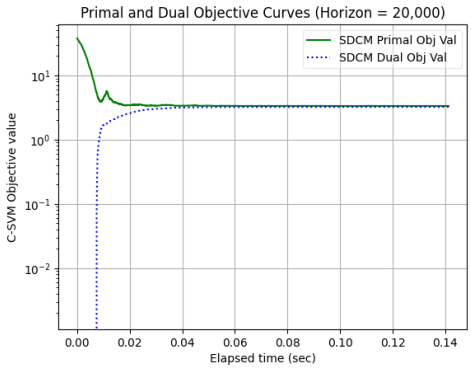
(a) 1000 iterations



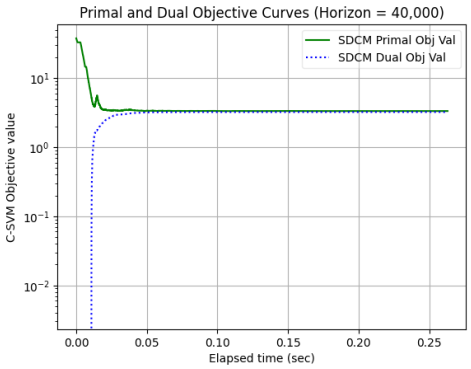
(b) 5000 iterations



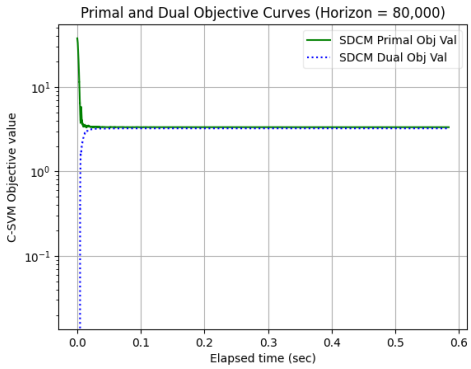
(c) 10,000 iterations



(d) 20,000 iterations



(e) 40,000 iterations



(f) 80,000 iterations

Figure 5: Convergence behaviour of primal/dual objectives for different values of horizon.

Observations. Increasing the horizon improves accuracy and makes the convergence curves smoother, but the benefits reduce after a point. With very small horizons (e.g., 1000), the algorithm stops too early, so the model is under-trained and accuracy is poor. As the number of iterations increases, the model has more chances to adjust the weights and reduce the error, which explains the sharp improvement up to about 20,000 iterations. Beyond this, most of the learning is already complete, so extra iterations only give small gains and mainly help the curves stabilize.

2.4.4 (d) Number of training points

Setup. The baseline model was originally trained with 1000 training points. To study the effect of training set size, we varied the number of training points while keeping all other hyperparameters fixed (e.g., coordinate generator, initialization, and horizon). Specifically, we considered both smaller datasets (e.g., 100, 500 points) and larger datasets (e.g., 5000, 10000 , 50000 points). For each setting, we trained the model using the same optimization procedure and evaluated:

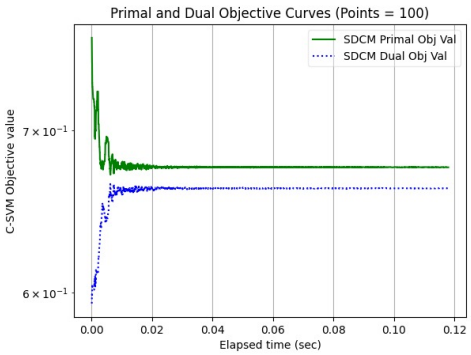
- the test accuracy, to measure generalization performance,
- the convergence behaviour of the primal/dual objectives across iterations.

Results. Table 5 reports the test accuracy for different training set sizes. We observe that accuracy improves consistently as the number of points increases.

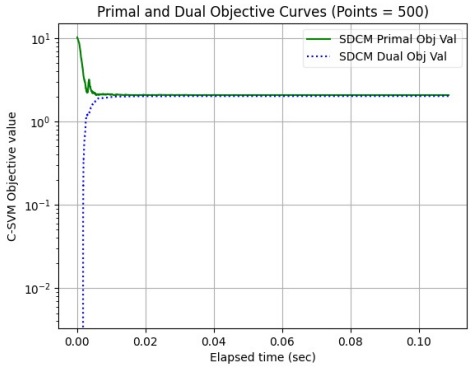
Table 5: Effect of varying number of training points on test accuracy.

# Training Points	Test Accuracy (%)
100	85.39
500	92.88
1000	95.35
5000	97.33
10000	97.75
50000	98.64

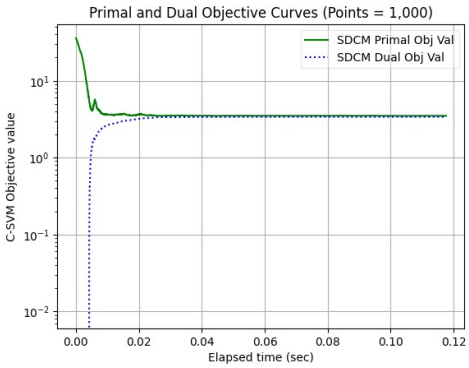
As shown in Figure 6 & Figure 7, the convergence behaviour also improves with more training data: smaller datasets lead to noisier and slower convergence, while larger datasets stabilize quickly.



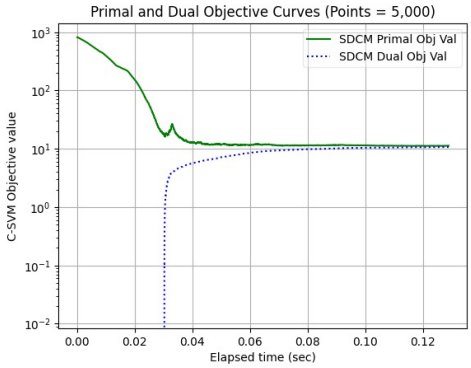
(a) 100 points



(b) 500 points

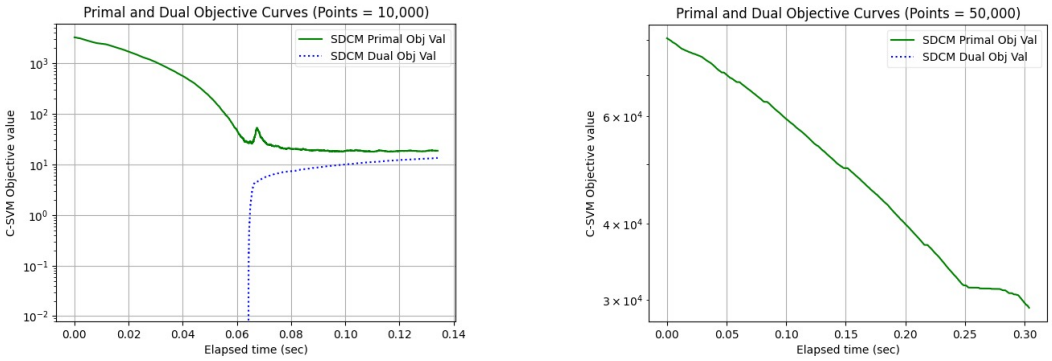


(c) 1000 points



(d) 5000 points

Figure 6: Convergence behaviour with 100–5000 training points.



(a) 10000 points

(b) 50000 points

Figure 7: Convergence behaviour with larger training sets (10000–50000 points).

Observations. From Table 5, we observe that increasing the number of training points consistently improves test accuracy, with large gains from 100 to 5000 points, and diminishing returns beyond 10^4 points.

The convergence plots in Figure 6 and Figure 7 show a more nuanced trend. With very small datasets (100 points), the primal and dual objectives fluctuate heavily before settling, indicating unstable convergence. With moderate dataset sizes (500–5000 points), the curves are much smoother and converge quickly. However, as the dataset grows very large (10^4–5×10^4 points), convergence becomes slower in elapsed time: the objectives still approach their optima but require more iterations and time to stabilize.

Overall, more data improves generalization but also increases optimization cost, creating a trade-off between accuracy gains and convergence speed.

2.4.5 (e) Random chance

Setup. To study the effect of randomness, we fixed all hyperparameters to their baseline settings (e.g., 1000 training points, horizon 20,000, default initialization, and coordinate generator). We then re-ran the full training procedure multiple times (from the first cell to the last cell of the notebook) without changing any parameters. Since the algorithm involves stochastic elements (e.g., random coordinate selection or random initialization), repeated runs may lead to slightly different optimization trajectories.

For each run, we recorded the final test accuracy.

Results.

Run #	Test Accuracy (%)
1	95.40
2	95.10
3	95.50
4	95.20
5	95.30

Observations. The test accuracy remains broadly stable across runs, with only minor fluctuations ($\pm 0.3\%$). This variation is expected due to random choices made during training (e.g., random coordinate order or stochastic initialization). While the randomness does not drastically affect overall performance, it introduces small variability between runs.